User Model, Mental Model, and Requirements Analysis for a Smart House Design & Cost Recommendation System

Luvo Goli • Lethabo Lethabo • Elam Mandulo • Lukhanyo Msothwane • Samkele Mthuli • Chuma Nzangabom

Group: FivePlusOne Devs

Department of Computer Science · CSC 312 Software Engineering · University of the Western Cape

ABSTRACT

This report documents Step 4 of the FivePlusOne Devs software engineering project: the User Model, Mental Model, and Requirements Analysis for a Smart House Design & Cost Recommendation System. Prospective homeowners often struggle to match house designs to their plot size and budget, resulting in confusion and unreliable cost estimates. We identify the system's end users and the tasks they must perform, describe the conceptual (mental) model the interface should evoke, and analyse the resulting data, functional and behavioural requirements. A UML use case diagram models user–system interaction, and a high-level UML class diagram models the underlying data and behaviour. This analysis extends the process, principles, methods, goals, umbrella activities and toolchain established in the project proposal (Step 2) and will directly inform the Design and Implementation Models produced in later steps.

1. Introduction

The FivePlusOne Devs team is developing a system that recommends suitable house designs and estimated construction costs to prospective homeowners, based on the plot size and budget they provide. Following the Agile Scrum process adopted in Step 2, this document reports the analysis activities of the current sprint: building a model of the system's users, capturing the mental model the interface should reinforce, and analysing the requirements that follow from both.

User needs were identified through a combination of informal discussions with prospective homeowners and family members about how they currently choose house plans and estimate building costs, first-hand consideration of the team's own experience researching house designs, and a review of comparable existing platforms such as property listing and show-home marketing sites and online mortgage/affordability calculators. These activities, together with the functional scope agreed upon in Step 2, informed the user profiles, use cases and requirements presented below.

2. User Model

2.1 User Profiles

The user model identifies who will use the system and the tasks each type of user should be able to perform. Two primary end-user roles interact with the system directly, while two secondary roles influence its content indirectly.

Primary: Homeowner / Client

- Register and log in to a personal account.
- Enter plot size and construction budget.
- Browse and search available house designs.
- View house plans recommended for their inputs.
- View the estimated construction cost of a design.

Primary: System Administrator

- Log in to an administrative account.
- Create, update, or remove user accounts.
- Add, edit, or remove house plan and cost data.

Secondary: Architects & House Designers

- Supply house designs and specifications that populate the system (via the administrator).

Secondary: Contractors

- Provide cost-estimation data used to calculate construction cost outputs (via the administrator).

2.2 Use Case Model

Figure 1 models the interactions between the two primary actors and the system. The Homeowner/Client registers or logs in, supplies plot and budget details, and browses designs, recommendations and cost estimates. The System Administrator manages user accounts and house-plan data, and must also authenticate. «Include» dependencies show that viewing recommendations always requires plot/budget input to already have been supplied, and that viewing an estimated cost always requires a recommended plan to have been viewed first.

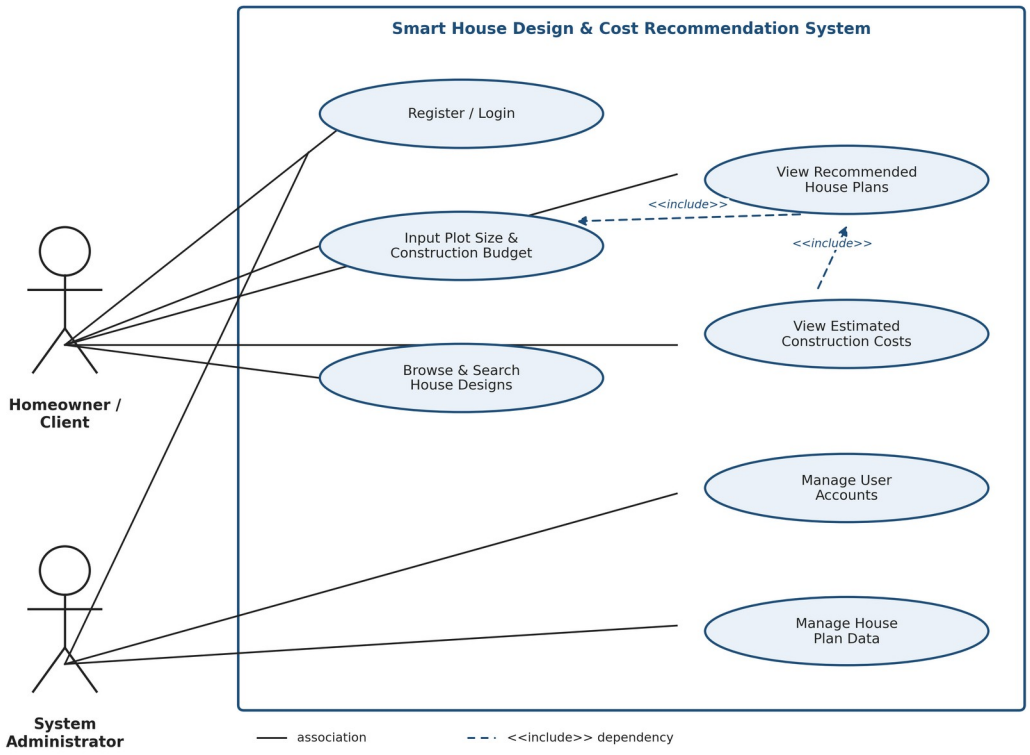


Figure 1. *UML use case diagram of the Smart House Design & Cost Recommendation System.*

3. Mental Model

The interface should evoke concepts homeowners already understand from familiar, real-world and digital experiences, rather than requiring them to learn new mental models.

- **Design catalogue / show-home village:** House designs are presented as browsable "cards" with an image, name and summary, mirroring property-listing sites and show-home brochures that users already know how to navigate.
- **Budget & plot filters:** Plot size and budget are entered as filter-style inputs, similar to price and size filters on e-commerce or property-search platforms, so narrowing down options feels familiar.
- **Quote / cost-calculator:** Estimated construction costs are presented the way an online mortgage or renovation-quote calculator presents a result: a clear, itemised total the user can trust and compare across options.
- **Guided wizard:** The plot-and-budget entry, recommendation and cost-estimate steps are presented as a short, linear wizard rather than a single complex form, matching the step-by-step flow of loan-application and checkout processes.

Reinforcing these familiar metaphors in the eventual UI design (Step 5) should reduce the learning curve and the confusion the project abstract identifies as the core problem.

4. Requirements Analysis

4.1 Data, Functional & Behavioural Requirements

Data requirements

- User account data: name, email, password (hashed), role.
- Client input data: plot size, construction budget.
- House design data: name, style, plot-size range, description, image.
- Cost data: material cost, labour cost, total estimated cost, linked to a design.

Functional requirements

- Users can register and log in securely.
- Clients can input plot size and construction budget.
- Clients can browse and search house designs.
- The system recommends house plans matching a client's inputs.
- The system displays an estimated construction cost per design.
- Administrators can manage user accounts and house-plan data.

Behavioural / non-functional requirements

- If no design matches the given plot size and budget, the system must display the closest alternatives with a clear notice rather than an empty result.

- Numeric inputs (plot size, budget) must be validated before recommendations are generated.
- Recommendation results must be returned promptly to keep the interaction responsive.
- The interface must remain usable on both desktop and mobile screen sizes.
- The system must remain reliably available, and the architecture must stay maintainable and scalable as designs and users grow.

4.2 Use Case Descriptions

The two use cases most central to the system's value proposition are described in detail below; the remaining use cases in Figure 1 follow the same structure.

Use Case: View Recommended House Plans

Actor	Homeowner / Client
Precondition	The client is logged in and has entered a plot size and construction budget.
Main flow	1. Client selects “View Recommendations.” 2. System r[illegible]ves designs whose plot-size range matches the client's plot size. 3. System f[illegible] those designs by the client's budget. 4. System displays the matching designs, ranked by best fit. 5. Client selects a design to view its estimated construction cost.
Alternate flow	If no design matches both plot size and budget, the system displays the closest alternatives and notifies the client that these fall outside one or both criteria.
Postcondition	The client has viewed a ranked list of house designs suited to their plot size and budget.

Use Case: Register / Login

Actor	Homeowner / Client, System Administrator
Precondition	The user has a valid email address (registration) or an existing account (login).
Main flow	1. User selects “Register” or “Login.” 2. User supplies the required credentials. 3. System validates the credentials and creates or authenticates the account. 4. System directs the user to the appropriate dashboard (client or administrator).
Alternate flow	If credentials are invalid or the email is already registered, the system displays a clear error and allows the user to retry.
Postcondition	The user holds an authenticated session appropriate to their role.

4.3 Class Model

Figure 2 models the system's core data and behaviour at a high, abstract level; further detail (e.g. multiplicities, method signatures) will be added during the Design step. Client and Administrator specialise an abstract User, reflecting their shared authentication behaviour and distinct responsibilities. A RecommendationEngine matches HouseDesign records to a client's inputs, and a CostEstimator derives a Cost from a chosen design.

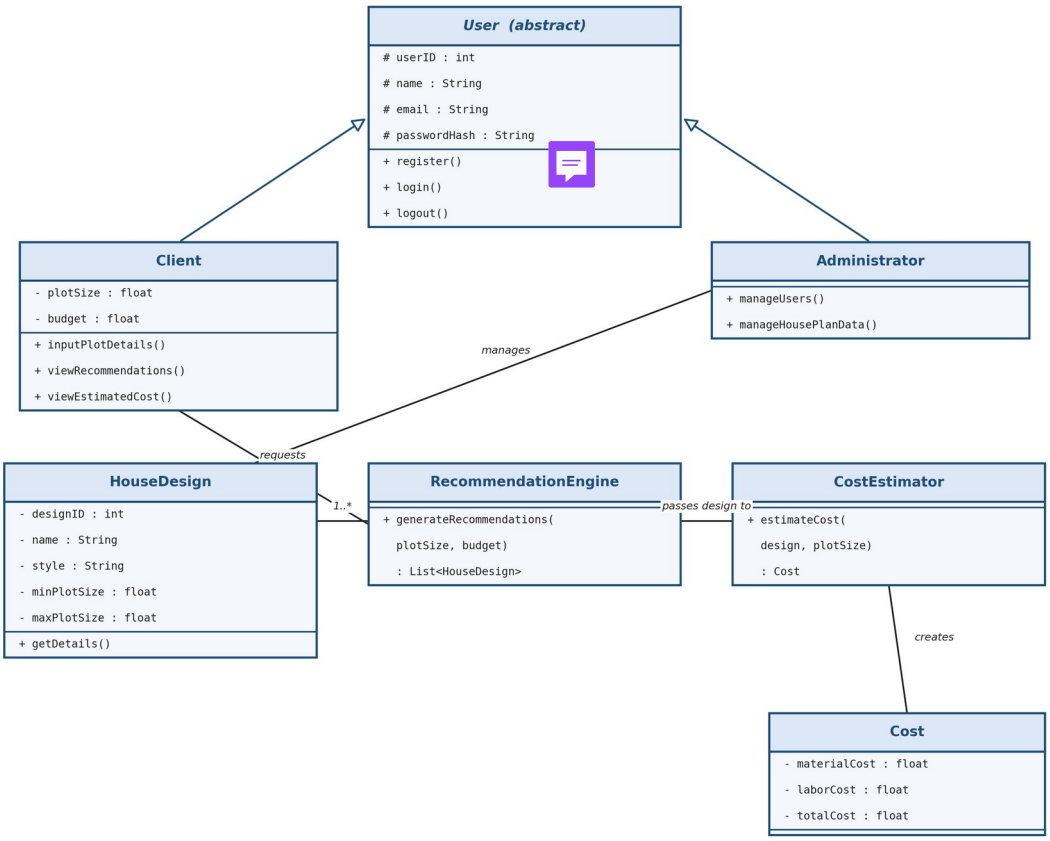


Figure 2. *High-level UML class diagram of the system's core domain model.*

5. Conclusion & Next Steps

This analysis establishes a clear picture of who will use the Smart House Design & Cost Recommendation System, the mental model the interface should reinforce, and the data, functional and behavioural requirements that follow. Together with the use case and class models, it provides a stable foundation for the next Scrum sprint, in which the team will produce the Design Model — translating these requirements into a detailed interface design, database schema and system architecture — followed by the Implementation Model. Quality assurance, risk management and version control will continue to be applied throughout, consistent with the software engineering practices established in Step 2 of this project.